

# CONVENTIONS DE NOMMAGE

EDLoc — règles de nommage de la base de données, de l'API, du code et du dépôt Git

## 1. Objet du document

Une convention de nommage est un ensemble de règles fixées pour nommer les éléments d'un projet de façon cohérente et prévisible : quiconque connaît la règle peut deviner un nom sans l'avoir jamais rencontré, et comprendre un nom sans documentation. Ces conventions réduisent la charge mentale, évitent les incohérences sources de bugs (par exemple une colonne nommée `idBien` à un endroit et `id_bien` à un autre) et rendent le projet lisible par un tiers. Le présent document fait foi : tout nouvel élément du projet (table, route, fichier, composant, branche) doit suivre les règles ci-dessous, qui sont celles déjà appliquées dans les livrables de conception (MPD, routes de l'API, arborescence du projet).

## 2. Règle de langue

Le vocabulaire métier est en français partout : noms de tables et de colonnes, ressources de l'API, libellés d'écrans, noms de fichiers métier. Cette règle garantit une correspondance directe entre le cahier des charges, le modèle de données, l'API et l'interface — le même mot désigne la même chose de bout en bout. L'anglais est réservé aux termes imposés par la technique : mots-clés des langages, noms des bibliothèques, fichiers imposés par les frameworks (`page.tsx`, `layout.tsx`, `schema.prisma`) et préfixes conventionnels de l'écosystème (use pour les hooks React, feat/fix pour les commits).

## 3. Base de données (PostgreSQL)

Le standard PostgreSQL est le `snake_case` (minuscules, mots séparés par des tirets bas). Les tables sont au singulier : une ligne représente un bien, un élément, une signature. Les exemples ci-dessous sont extraits du MPD du projet (fichier `edloc_mpd.sql`).

| Élément       | Convention                                         | Exemples (EDLoc)                                         |
|---------------|----------------------------------------------------|----------------------------------------------------------|
| Table         | singulier · snake_case · français                  | <code>bailleur · bien · etat_des_lieux · piece</code>    |
| Colonne       | snake_case, nom explicite                          | <code>mot_de_passe_hashe · date_creation</code>          |
| Clé primaire  | <code>id_&lt;table&gt;</code> · UUID               | <code>id_bien · id_edl</code> (forme abrégée admise)     |
| Clé étrangère | même nom que la clé référencée                     | <code>id_bailleur</code> dans la table <code>bien</code> |
| Contrainte FK | <code>fk_&lt;table&gt;_&lt;référence&gt;</code>    | <code>fk_bien_bailleur · fk_piece_edl</code>             |
| Type énuméré  | <code>&lt;nom&gt;_enum</code> · valeurs snake_case | <code>statut_edl_enum ('brouillon','signe')</code>       |

## 4. API REST

Les URL désignent des ressources, jamais des actions : le verbe est porté par la méthode HTTP (GET consulte, POST crée, PUT/PATCH modifie, DELETE supprime). Les ressources sont au pluriel — l'URL désigne la collection, là où la table désigne une ligne : la table `bien` (singulier) est exposée via `/api/biens` (pluriel). Les mots composés utilisent le kebab-case (tirets), lisible dans une URL.

| Élément              | Convention                        | Exemples (EDLoc)                                  |
|----------------------|-----------------------------------|---------------------------------------------------|
| Ressource            | pluriel · kebab-case · français   | <code>/api/biens · /api/etats-des-lieux</code>    |
| Identifiant          | paramètre <code>:id</code> (UUID) | <code>/api/pieces/:id</code>                      |
| Création d'un enfant | imbriquée sous le parent          | <code>POST /api/etats-des-lieux/:id/pieces</code> |

| Élément             | Convention                   | Exemples (EDLoc)                         |
|---------------------|------------------------------|------------------------------------------|
| Ressource existante | adressée à plat par son UUID | PUT /api/pièces/:id                      |
| Action non CRUD     | sous-ressource nominale      | GET /api/etats-des-lieux/:id/comparaison |
| Préfixe de zone     | selon le niveau d'accès      | /api/auth/... · /api/admin/...           |

La règle d'imbrication superficielle (création imbriquée, ressource existante à plat) est détaillée et justifiée dans le document « Arborescence du site & routes de l'API ».

## 5. Code TypeScript (Backend et Frontend)

Le code suit les conventions de l'écosystème TypeScript/React, avec le vocabulaire métier en français : la variable `etatDesLieux` correspond à la table `etat_des_lieux` et à la ressource `/api/etats-des-lieux` — seule la casse change selon le contexte.

| Élément                 | Convention                                         | Exemples (EDLoc)                                       |
|-------------------------|----------------------------------------------------|--------------------------------------------------------|
| Variable, fonction      | camelCase                                          | <code>etatDesLieux · genererPdf()</code>               |
| Type, interface         | PascalCase                                         | <code>Bien · EtatDesLieux · RoleSignataire</code>      |
| Constante globale       | UPPER_SNAKE_CASE                                   | <code>TAILLE_MAX_PHOTO</code>                          |
| Fichier Backend         | <code>&lt;ressource&gt;.&lt;rôle&gt;.ts</code>     | <code>biens.routes.ts · edl.service.ts</code>          |
| Middleware              | <code>&lt;fonction&gt;.&lt;précision&gt;.ts</code> | <code>auth.jwt.ts · role.admin.ts · validate.ts</code> |
| Schéma Zod              | <code>&lt;action ressource&gt;Schema</code>        | <code>creerBienSchema · connexionSchema</code>         |
| Composant React         | PascalCase.tsx                                     | <code>CarteBien.tsx · PastilleEtat.tsx</code>          |
| Hook React              | <code>use + PascalCase</code>                      | <code>useAuth · useEtatDesLieux</code>                 |
| Dossier / route Next.js | kebab-case · groupes entre parenthèses             | <code>tableau-de-bord/ · (bailleur)/</code>            |

## 6. Dépôt Git

Le dépôt suit la convention Conventional Commits : chaque message commence par un type qui décrit la nature du changement, ce qui rend l'historique lisible et filtrable. Les branches de travail sont préfixées par leur objet.

| Élément                   | Convention                                    | Exemples (EDLoc)                         |
|---------------------------|-----------------------------------------------|------------------------------------------|
| Branche principale        | main                                          | main                                     |
| Branche de fonctionnalité | feature/< sujet-kebab-case >                  | feature/signature-tactile                |
| Branche de correction     | fix/< sujet-kebab-case >                      | fix/comparaison-ecarts                   |
| Commit — fonctionnalité   | feat: <description à l'infinitif ou nominale> | feat: ajout de la signature du locataire |
| Commit — correction       | fix: <description>                            | fix: correction du calcul des écarts     |
| Commit — autres types     | docs: · refactor: · test: · chore:            | docs: mise à jour du README              |

## 7. Application de ces règles

- Ces conventions sont déjà appliquées dans les livrables existants : le MPD (tables, contraintes, énumérations), le tableau des routes de l'API et l'arborescence du projet leur servent de référence croisée.
- Tout nouvel élément (entité, route, écran, composant, branche) doit s'y conformer ; en cas de doute, ce document fait foi.

- Une exception ponctuelle est possible si un outil l'impose (fichier généré, nom requis par un framework) : elle doit rester localisée et, si besoin, être commentée dans le code.